ΚΕΦΑΛΑΙΟ 11 - ΣΧΕΔΙΑΣΗ

Δύο “όψεις” για κάθε προϊόν λογισμικού

- Actions
- Data

Δύο βασικοί τρόποι για σχεδίαση ενός προϊόντος

- Action-oriented σχεδίαση
- Data-oriented σχεδίαση
- Μειονέκτημα τους : δε λαμβάνουν υπόψιν το ένα εκ των δύο συστατικών μερών του λογισμικού

Τρίτος τρόπος (λύση)

- Υβριδικές μέθοδοι : ίδια σημασία και σε actions και σε data - π.χ. αντικειμενοστρεφής σχεδίαση

ΔΡΑΣΤΗΡΙΟΤΗΤΕΣ ΣΤΗ ΦΑΣΗ ΤΗΣ ΣΧΕΔΙΑΣΗΣ

Η φάση σχεδίασης αποτελείται από:

- Αρχιτεκτονική Σχεδίαση
- Αναλυτική Σχεδίαση
- Έλεγχο Σχεδίασης

Αρχιτεκτονική Σχεδίαση (architectural or logical or high-level design):

- Input: προδιαγραφές (περιγραφή του τι θα κάνει το προϊόν)
- Output: πώς θα το κάνει
  → τμήματα από τα οποία το προϊόν απαρτίζεται (modular decomposition)
- Abstraction: γίνεται υπόθεση ότι τα τμήματα υπάρχουν

Αναλυτική Σχεδίαση (detailed or modular or low-level design):

- Σχεδίαση κάθε τμήματος με λεπτομέρεια
  - συγκεκριμένοι αλγόριθμοι
  - δομές δεδομένων (data structures)
- Abstraction: η διασύνδεση των τμημάτων αγνοείται εδώ

ACTION-ORIENTED ΜΕΘΟΔΟΙ ΣΧΕΔΙΑΣΗΣ

Στόχος: Διάσπαση σε σπονδύλους (modules) με ψηλή συνεκτικότητα (high cohesion) και χαμηλή σύζευξη (low coupling)

Δύο τεχνικές :

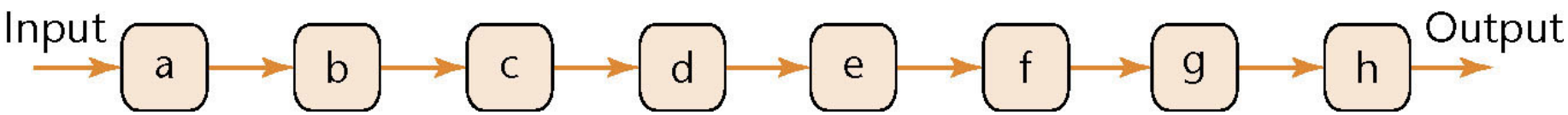
1. **Ανάλυση Ροής Δεδομένων (ΑΡΔ) - Data Flow Analysis (DFA)**
2. **Ανάλυση Διεργασιών (ΑΔ) – Transaction Analysis (TA)**

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ

- Χρησιμοποιείται σε συνδυασμό με τις περισσότερες τεχνικές καθορισμού προδιαγραφών (π.χ. Structured Systems Analysis - SSA)
- Οδηγεί σε επίτευξη υψηλής συνεκτικότητας
- Το input της τεχνικής αυτής είναι ένα ΔΡΔ

SSA: Το ΔΡΔ παρέχει πλήρεις και ακριβείς πληροφορίες για το input και output κάθε τμήματος του προϊόντος

Αναπαράσταση ροής δεδομένων μέσω ΔΡΔ:

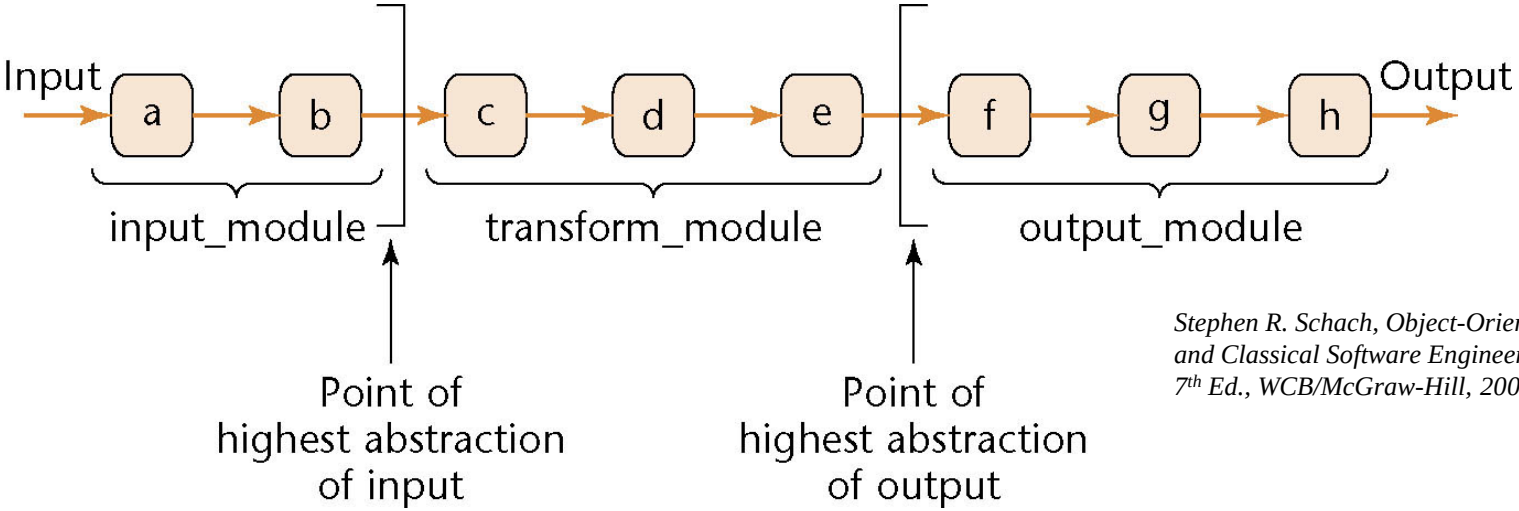


ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ (...συνέχεια)

Παράδειγμα: Το προϊόν μετατρέπει input σε output

"Σημείο υψηλότερου abstraction του input" (παύει να είναι input)

"Σημείο υψηλότερου abstraction του output" (αρχίζει το output)



Stephen R. Schach, Object-Oriented and Classical Software Engineering, 7th Ed., WCB/McGraw-Hill, 2007

Διαίρεση του προϊόντος σε τρία modules με βάση τα points of highest abstraction

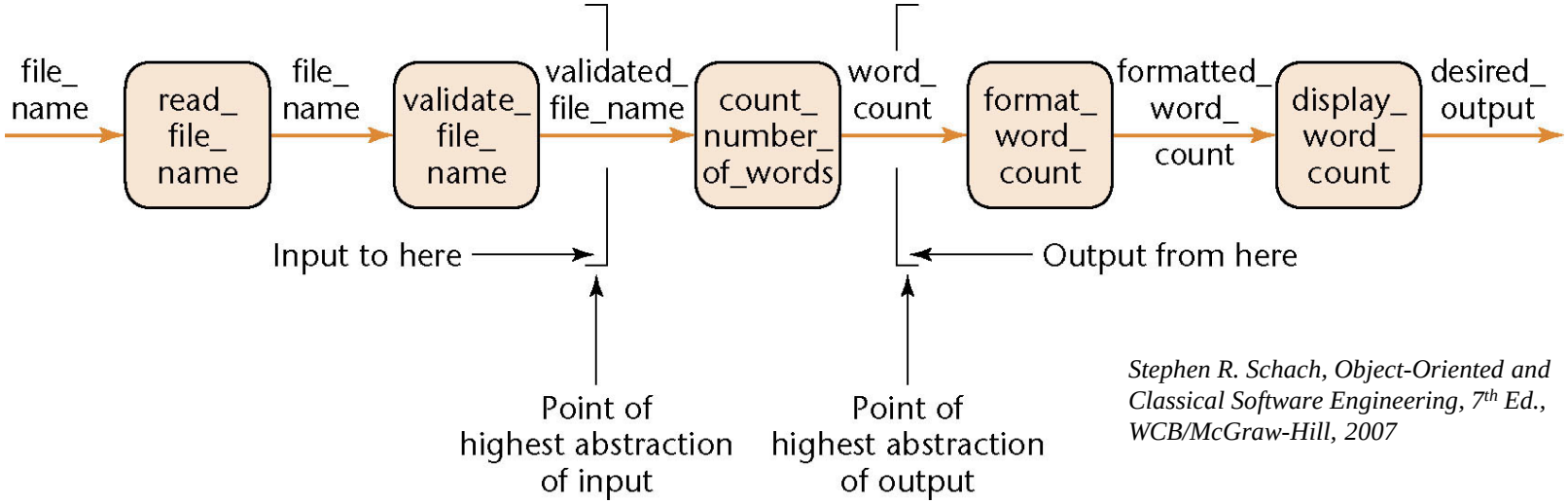
- input module
- transform module
- output module

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ (...συνέχεια)

Επανάληψη (stepwise) αποσύνθεσης κάθε module με βάση τα σημεία abstraction του μέχρις ότου να έχει υψηλή συνεκτικότητα (η ΑΡΔ δεν αποσκοπεί σε χαμηλή σύζευξη – πιθανώς να χρειαστούν μικροτροποποιήσεις)

Παράδειγμα:

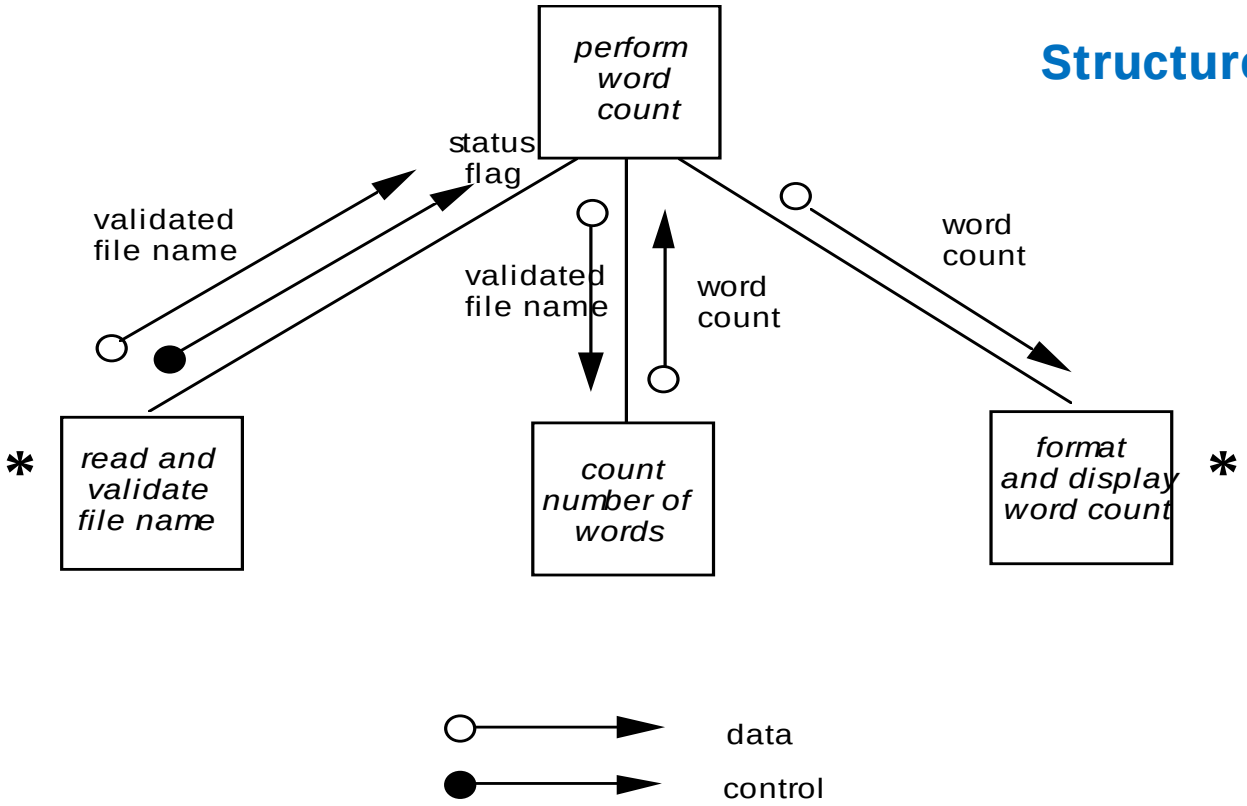
- Σχεδίαση προϊόντος που να δέχεται input ένα όνομα αρχείου, και επιστρέφει τον αριθμό των λέξεων του αρχείου αυτού (UNIX wc)



Stephen R. Schach, Object-Oriented and Classical Software Engineering, 7th Ed., WCB/McGraw-Hill, 2007

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ - ΠΑΡΑΔΕΙΓΜΑ

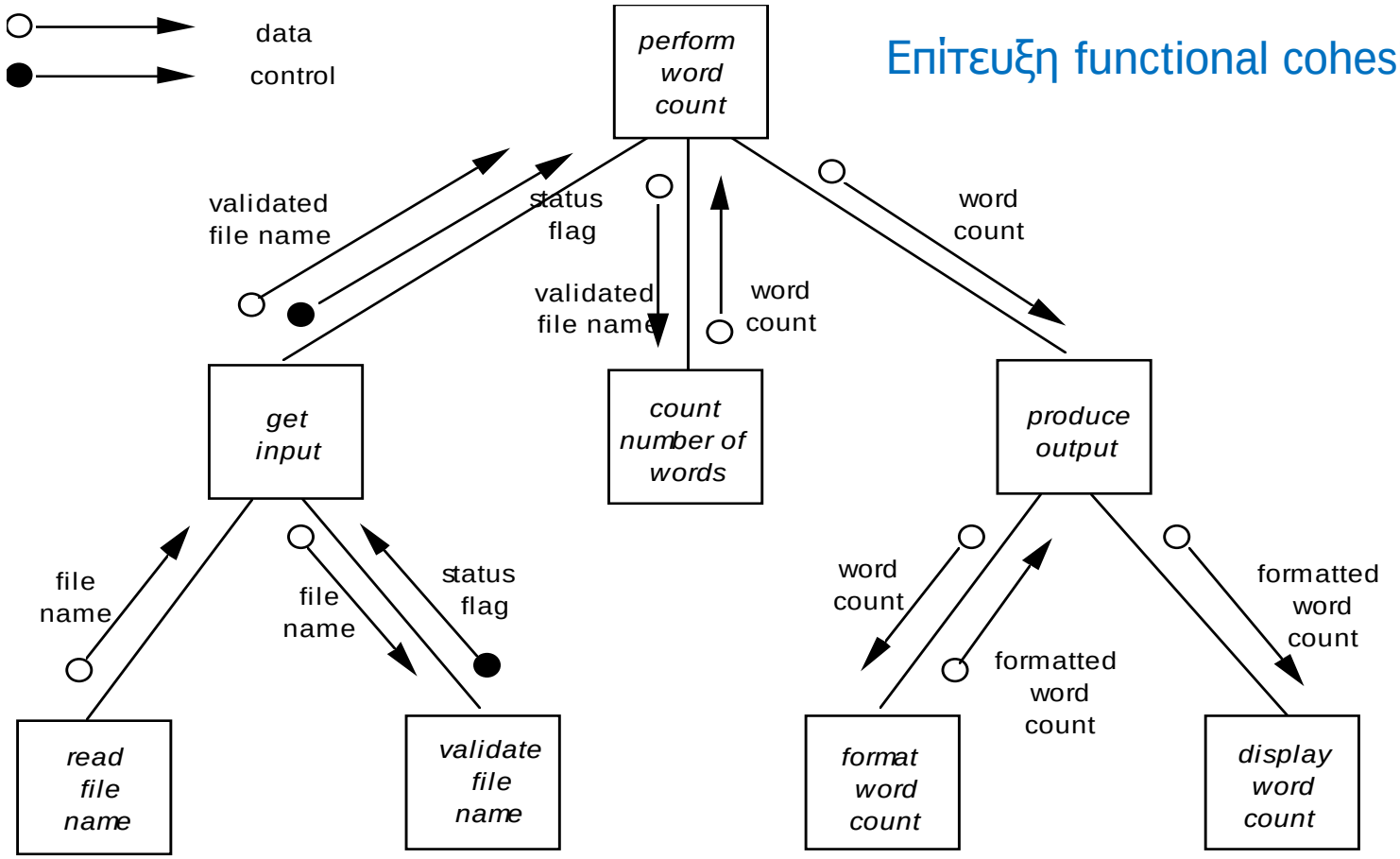
Πρώτο πέρασμα (δες αρχικό ΔΡΔ, ανάγκη για "ροή ελέγχου")



- Ροή δεδομένων υπό κάποια συνθήκη → ανάγκη για έλεγχο ροής
- Επιπλέον ανάλυση των modules με communicational cohesion (*)

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ - ΠΑΡΑΔΕΙΓΜΑ

Δεύτερο πέρασμα



Επίτευξη functional cohesion

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ - ΠΑΡΑΔΕΙΓΜΑ

Αναλυτική Σχεδίαση Σπονδύλων, Πίνακας

Module name	**read_file_name**
Module type	Function
Return type	**string**
Input arguments	None
Output arguments	None
Error messages	None
Files accessed	None
Files changed	None
Modules called	None
Narrative	The product is invoked by the user by means of the command string **word_count <file_name>** Using an operating system call, this module accesses the contents of the command string input by the user, extracts **<file_name>**, and returns it as the value of the module.

Stephen R. Schach, Object-Oriented and Classical Software Engineering, 7th Ed., WCB/McGraw-Hill, 2007

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ - ΠΑΡΑΔΕΙΓΜΑ
Αναλυτική Σχεδίαση Σπονδύλων, Πίνακας (… συνέχεια)

Module name	**validate_file_name**
Module type	Function
Return type	**Boolean**
Input arguments	**file_name : string**
Output arguments	None
Error messages	None
Files accessed	None
Files changed	None
Modules called	None
Narrative	This module makes an operating system call to determine whether file **file_name** exists. The module returns **true** if the file exists and **false** otherwise.

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ - ΠΑΡΑΔΕΙΓΜΑ
Αναλυτική Σχεδίαση Σπονδύλων, Πίνακας (… συνέχεια)

Module name	**produce_output**
Module type	Function
Return type	**void**
Input arguments	**word_count : integer**
Output arguments	None
Error messages	None
Files accessed	None
Files changed	None
Modules called	**format_word_count** arguments: **word_count : integer** **formatted_word_count : string** **display_word_count** arguments: **formatted_word_count : string**
Narrative	This module takes the integer **word_count** passed to it by the calling module and calls **format_word_count** to have that integer formatted according to the specifications. Then it calls **display_word_count** to have the line printed.

ΑΝΑΛΥΣΗ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ - ΠΑΡΑΔΕΙΓΜΑ

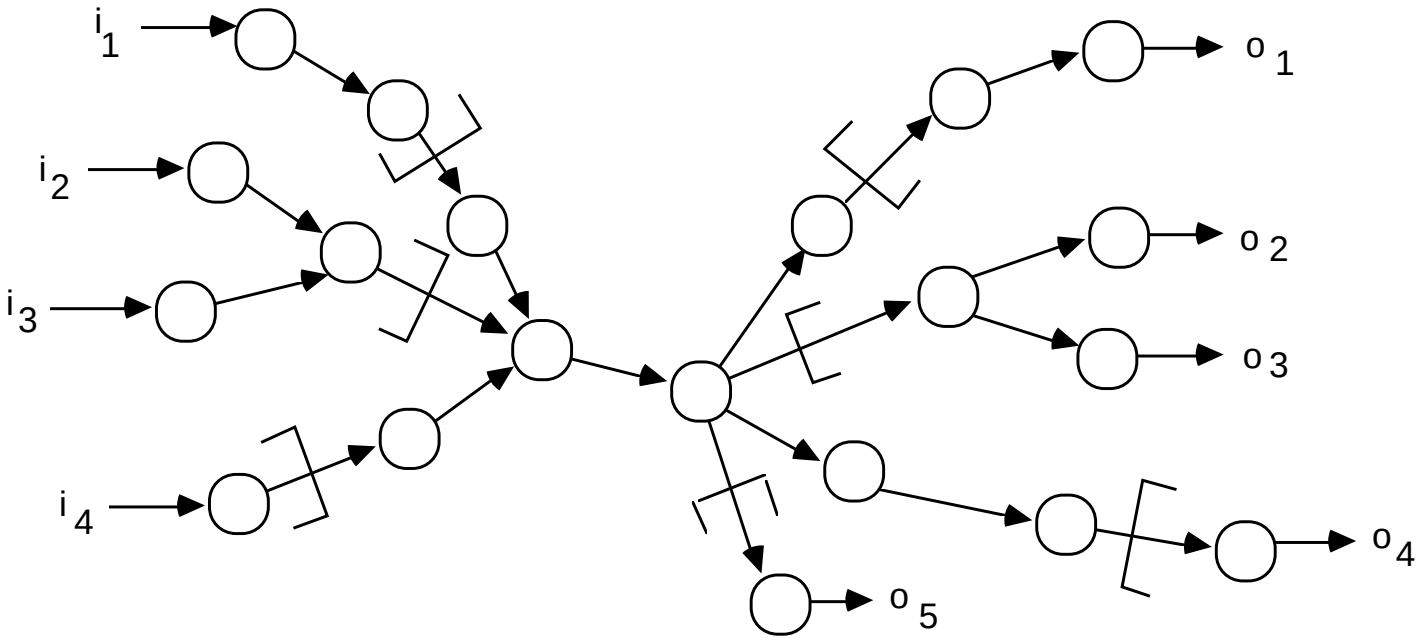
Αναλυτική Σχεδίαση Σπονδύλων - Ψευδοκώδικας

```
void perform_word_count()
{
  String                validate_file_name;
  int                   word_count;

  if (get_input (validate_file_name) is false)
     print "error 1: file does not exist";
  else
  {
    set word_count equal to count_number_of_words (validate_file_name);
    if (word_count is equal to –1)
       print "error 2: file is not a text file";
    else
      produce_output (word_count);
  }
}

Boolean get_input (String validate_file_name)
{
  String                file_name;

  file_name = read_file_name ();
  if (validate_file_name (file_name) is true)
  {
     set validate_file_name equal to file_name;
     return true;
  }
  else
    return false;
}

void display_word_count (String formatted_word_count)
{
  print formatted_word_count, left justified;
}

String format_word_count (int word_count);
{
  return "File contains" word_count "words";
}
```

Stephen R. Schach, Object-Oriented and Classical Software Engineering, 7th Ed., WCB/McGraw-Hill, 2007

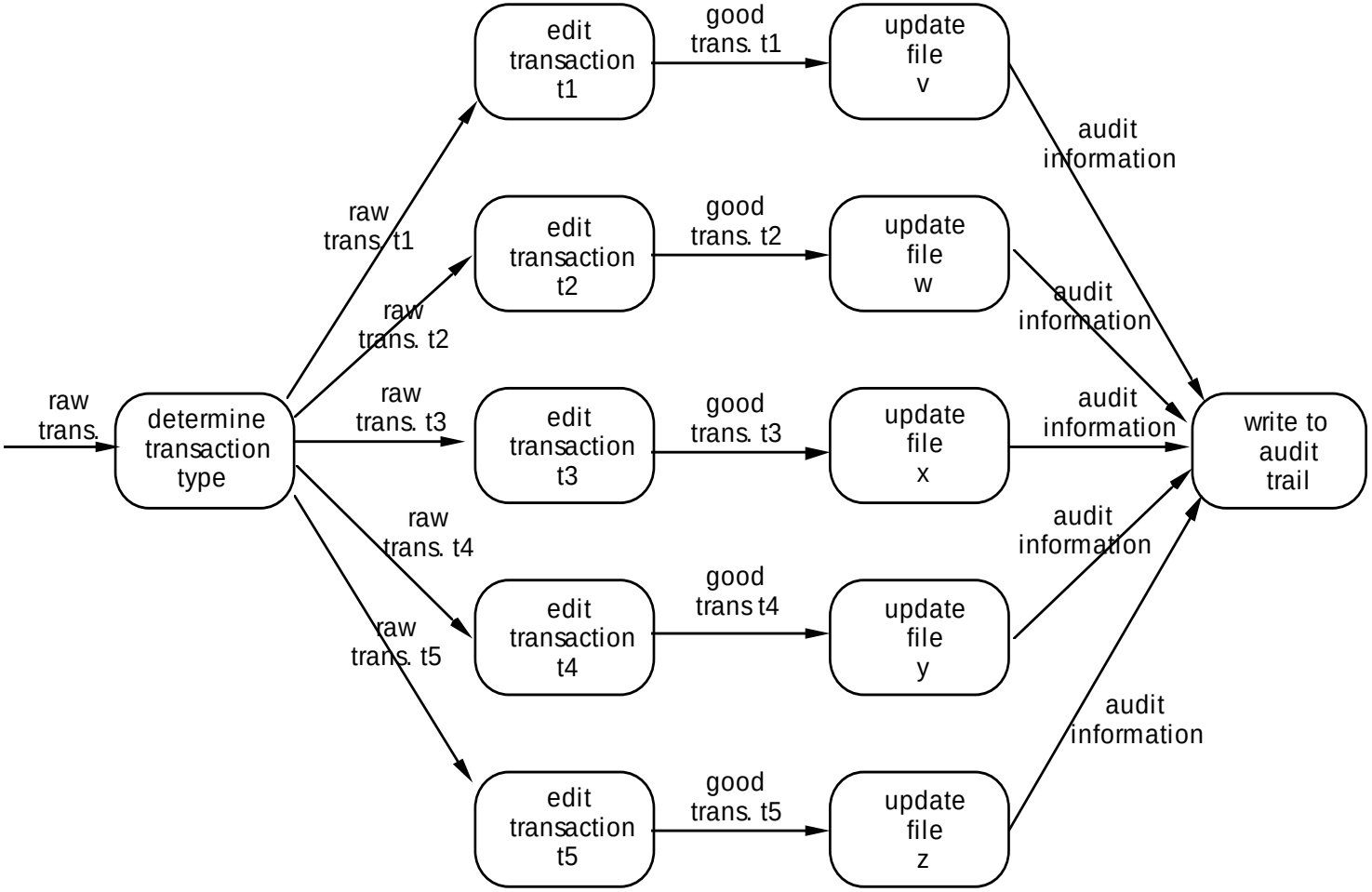
ΔΡΔ ΜΕ ΠΟΛΛΑΠΛΑ INPUT ΚΑΙ OUTPUT "ΚΑΝΑΛΙΑ"

- Πιο κοντά στην πραγματικότητα
- Εύρεση σημείων υψηλότερου abstraction για κάθε "κανάλι" από μια είσοδο σε κάποια έξοδο
- Δουλεύουμε όπως προηγουμένως με αποσύνθεση κάθε καναλιού σε modules με υψηλή συνεκτικότητα (και προσαρμόζουμε τη σύζευξη, αν χρειάζεται)

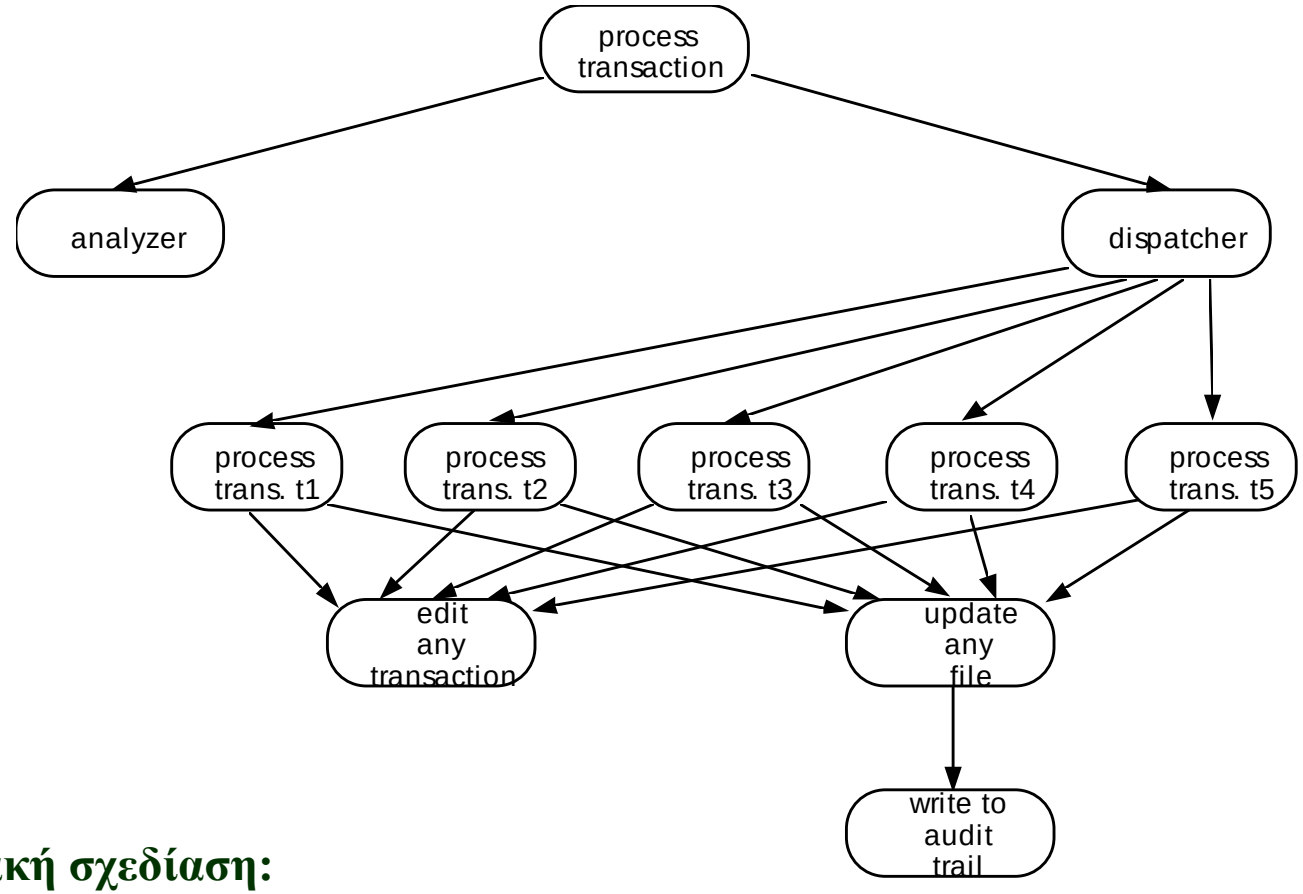


TRANSACTION ΑΝΑΛΥΣΗ

Η ΑΡΔ είναι ακατάλληλη για προϊόντα που αφορούν την επεξεργασία transactions (π.χ., για ένα ATM)



ΣΧΕΔΙΑΣΗ ΜΕΣΩ TRANSACTION ΑΝΑΛΥΣΗΣ



Κακή σχεδίαση:

Logical cohesion και control coupling (modules *edit any* και *update any*)

Λύση → Επαναχρησιμοποίηση λογισμικού: ένα βασικό module (edit ή update) και πολλαπλά στιγμιότυπά του (copies)

DATA-ORIENTED ΣΧΕΔΙΑΣΗ

Βασική αρχή

- Το προϊόν σχεδιάζεται σύμφωνα με τη δομή των δεδομένων του

Τρείς βασικές τεχνικές (παρόμοιες)

- Warnier [1976]
- Orr [1981]
- Jackson [1975]

Data-oriented σχεδίαση

- Δεν ήταν ποτέ τόσο δημοφιλής όσο η action-oriented σχεδίαση
- Παραμερίστηκε σημαντικά με την εμφάνιση της αντικειμενοστρεφούς σχεδίασης

ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΕΦΗΣ ΣΧΕΔΙΑΣΗ

Στόχος

- Σχεδίαση του προϊόντος κάνοντας χρήση των objects που αποφασίστηκαν στη φάση της Ο-Ο ανάλυσης

Αν χρησιμοποιείται γλώσσα που δεν υποστηρίζει inheritance (C, Ada 83) → πρέπει να γίνει abstract data type design (φτάνει η γλώσσα να υποστηρίζει **type** statements)

Αν χρησιμοποιείται γλώσσα χωρίς **type** δηλώσεις (FORTRAN, COBOL) → πρέπει να γίνει data encapsulation (data structures με όλα τα actions πάνω σε αυτά στο ίδιο module)

Αντικειμενοστρεφής σχεδίαση σε τέσσερα βήματα:

1. Κατασκευή interaction diagrams για κάθε σενάριο
2. Κατασκευή αναλυτικών class diagrams
3. Σχεδίαση του προϊόντος μέσω “clients of objects”
4. Αναλυτική σχεδίαση

ΠΡΟΒΛΗΜΑ ΑΝΕΛΚΥΣΤΗΡΑ: Ο-Ο ΣΧΕΔΙΑΣΗ

ΒΗΜΑ 1. ΚΑΤΑΣΚΕΥΗ INTERACTION DIAGRAMS ΓΙΑ ΚΑΘΕ ΣΕΝΑΡΙΟ

- sequence diagrams
- collaboration diagrams
- και τα δύο δείχνουν το ίδιο πράγμα, objects και messages που ανταλλάσσονται ανάμεσά τους, αλλά με διαφορετικό τρόπο

Τα sequence diagrams δίνουν έμφαση στην ακριβή χρονολογική ακολουθία των messages → χρήσιμα για τη σειρά εμφάνισης των events

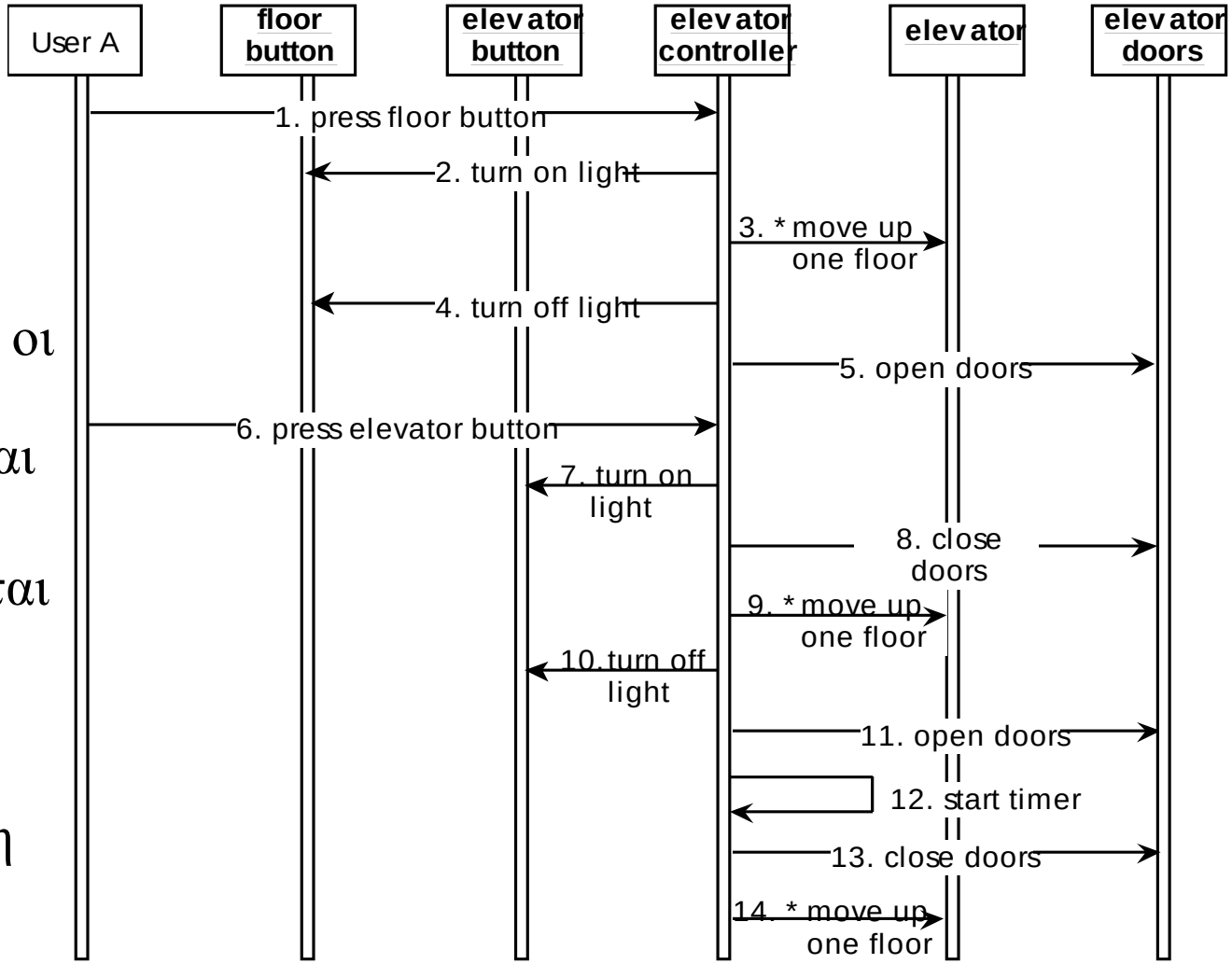
Τα collaboration diagrams δίνουν έμφαση στις σχέσεις ανάμεσα στα objects → χρήσιμα για την κατανόηση της δομής του προϊόντος

Παράδειγμα – Σενάριο ανελκυστήρα:

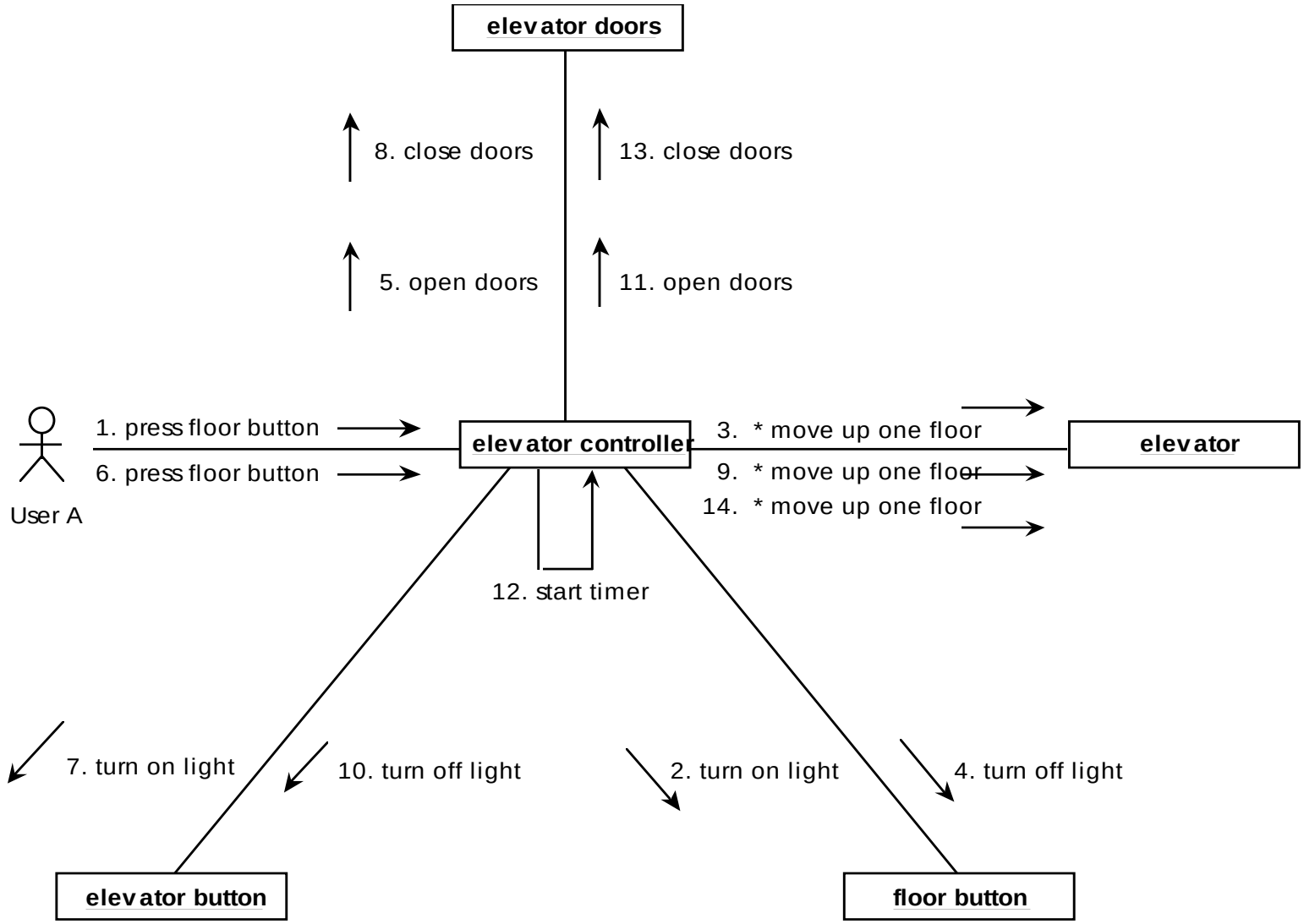
1. User A presses Up floor button at floor 3 to request elevator. User A wishes to go to floor 7.
2. Up floor button is turned on.
3. An elevator arrives at floor 3. It contains User B who has entered the elevator at floor 1 and pressed the elevator button for floor 9.
4. Up floor button is turned off.
5. Elevator doors open. User A enters elevator.
6. User A presses elevator button for floor 7.
7. Floor 7 elevator button is turned on.
8. Elevator doors close.
9. Elevator travels to floor 7.
10. Floor 7 elevator button is turned off.
11. Elevator doors open to allow User A to exit elevator.
12. Timer starts. User A exits.
13. Elevator doors close after timeout.
14. Elevator proceeds to floor 9 with User B

Sequence διάγραμμα

- Users και objects αναπαριστώνται με κουτιά και κάθετες γραμμές (χρόνος) – οι χρήστες μπορεί να αναπαρασταθούν και με γραφικό τρόπο
- Ο χρόνος εξελίσσεται από πάνω προς τα κάτω
- Το * (βλ. 3, 9, 14) δηλώνει επανάληψη



Collaboration διάγραμμα



ΒΗΜΑ 2. ΚΑΤΑΣΚΕΥΗ ΑΝΑΛΥΤΙΚΩΝ CLASS DIAGRAMS

Καθόρισε τα actions (“methods”) για κάθε class

- Πρόβλημα: ποιός θα κάνει ένα action, ένα class ή ένας “client” αυτής της class?
- Client ενός object είναι ένα unit που στέλνει message στο object

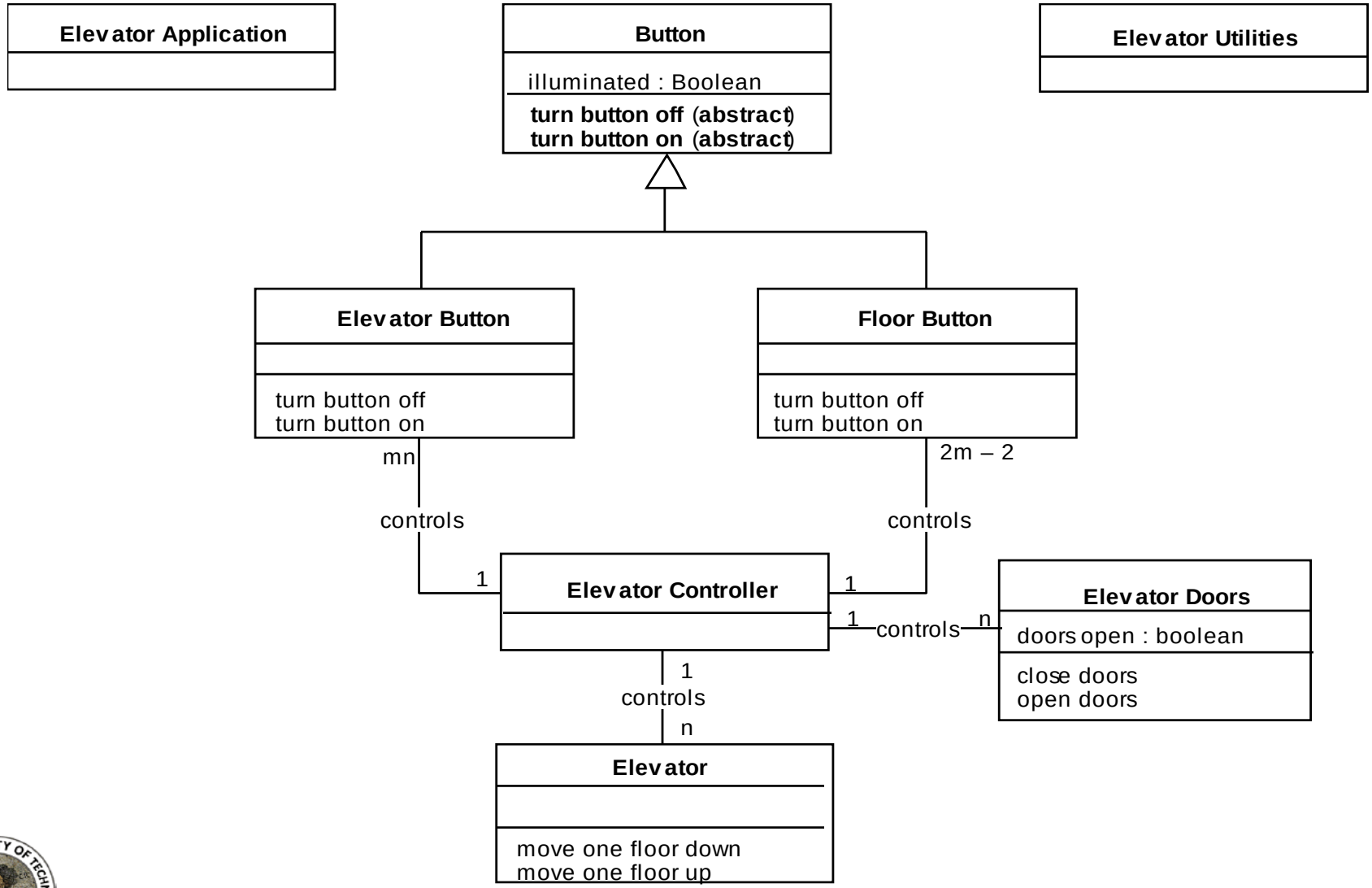
Κριτήρια

- Απόκρυψη πληροφορίας
- Μείωση αριθμού των actions : μέθοδος σε μια κλάση και κλήση της από τους clients
- Responsibility-driven σχεδίαση : ένας client που στέλνει ένα message σε ένα object δε χρειάζεται να ξέρει πώς, μόνο αν έχει εκτελεστεί το request. Η ευθύνη του τρόπου εκτέλεσης ανήκει στο object.

Παράδειγμα

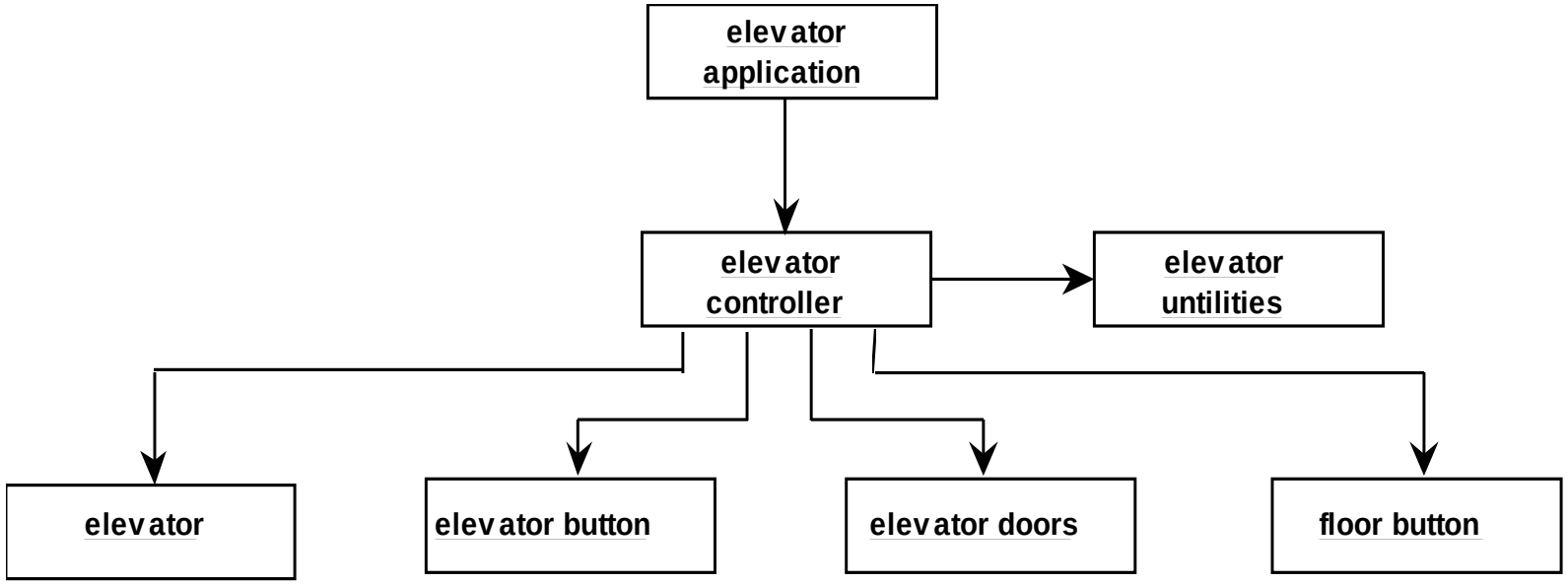
- Client του **Elevator Doors** (ένα object του **Elevator Controller**) → message to object of class **Elevator Doors** to close doors

ΒΗΜΑ 2. ΚΑΤΑΣΚΕΥΗ ΑΝΑΛΥΤΙΚΩΝ CLASS DIAGRAMS (...συνέχεια)



ΒΗΜΑ 3. ΣΧΕΔΙΑΣΗ ΤΟΥ ΠΡΟΪΟΝΤΟΣ ΜΕΣΩ "CLIENTS OF OBJECTS"

- Σχεδίασε ένα βέλος από τον "client" ενός object σε αυτό το object
- Objects που δεν έχουν "clients" πρέπει να γίνουν initiated (συνήθως) από το κυρίως πρόγραμμα (π.χ. την main method στη Java)



ΒΗΜΑ 3. ΣΧΕΔΙΑΣΗ ΤΟΥ ΠΡΟΪΟΝΤΟΣ ΜΕΣΩ "CLIENTS OF OBJECTS" (...συνέχεια)

Πιθανώς, ανάγκη για επιπλέον methods

Π.χ., για την class **Elevator Controller**

- Δεν έχει client κάποιο object (δηλ. Δεν καλείται από άλλο object) → Γενική τακτική: καλείται από τη *main* function (αρχικοποίηση προγράμματος)
- Πρέπει να καταγράφει και να ενημερώνει τα "requests"
- Χρειάζεται κάποιο state variable requests
- Χρειάζεται κάποια methods για καταγραφή των requests
  - log request
  - update requests
  - request is pending

ΒΗΜΑ 4. ΑΝΑΛΥΤΙΚΗ ΣΧΕΔΙΑΣΗ (για τη μέθοδο elevator controller loop που καλεί η main)

```
void elevator control loop () {
    do {
        if (a button has been pressed)
          if (button is not on) {
                button.turn button on;
                log request;
        }
        else if (elevator is moving up) {
          if (there is no request to stop at floor f)
                elevator.move one floor up;
          else {
                stop elevator by not sending a message to move;
                elevator doors.open doors;
                if (elevator button is on)
                      elevator button.turn button off;
```

```
                update requests;
                }
        }
        else if (elevator is moving down)
                [similar to up case]
        else if (elevator is stopped and request is pending) {
                elevator doors.close doors;
                if (floor button is on)
                        floor button.turn button off;
                determine direction of next request;
                elevator.move one floor up/down;
        }
        else if (elevator is at rest and not (request is pending))
                elevator doors.close doors;
        else    there are no requests, elevator is stopped with elevator
                        doors closed, so do nothing;
} }
```

ΣΧΕΔΙΑΣΗ REAL-TIME ΣΥΣΤΗΜΑΤΩΝ

Δυσκολίες

- Inputs από real world
  - το λογισμικό δεν μπορεί να έχει έλεγχο στο πότε φθάνουν τα inputs
- Συχνά υλοποιούνται σε distributed hardware
  - επιπλοκές στην επικοινωνία κατανεμημένων πόρων
  - ζητήματα timing
  - προβλήματα συγχρονισμού (π.χ. deadlock κλπ.)

Η μεγαλύτερη δυσκολία στη σχεδίαση real-time συστημάτων

- να ληφθούν υπόψιν οι χρονικοί περιορισμοί

Η σχεδίαση, ως μέρος του software process, δεν έχει κάνει την ίδια καλή εξελικτική πορεία για real-time systems όπως π.χ. η φάση των προδιαγραφών

ΜΕΘΟΔΟΙ ΣΧΕΔΙΑΣΗΣ REAL-TIME ΣΥΣΤΗΜΑΤΩΝ

Συνήθως, επεκτάσεις των nonreal-time μεθόδων

- Structured Development for Real-time Systems [Ward & Mellor, 1985]
- DARTS (Design Approach for Real-Time Systems) [Gomaa, 1986]
- SCR (Software Cost Reduction) [Kmielcik et al., 1984]

Δεν υπάρχει ικανοποιητική εμπειρία στη χρήση καθώς και το επιθυμητό επίπεδο ανάπτυξης τέτοιων μεθόδων

ΕΛΕΓΧΟΣ ΚΑΤΑ ΤΗ ΦΑΣΗ ΤΗΣ ΣΧΕΔΙΑΣΗΣ

- Η σχεδίαση πρέπει να λαμβάνει σωστά υπόψιν τις προδιαγραφές
- Σωστή ανάπτυξη της ίδιας της φάσης (όχι logic or interface faults)
- Design inspections and walkthroughs
- Transaction-driven και specification-driven έλεγχοι (επιθεωρήσεις)

CASE TOOLS ΓΙΑ ΤΗ ΦΑΣΗ ΤΗΣ ΣΧΕΔΙΑΣΗΣ

Front end or Upper-CASE tools

- Βασίζονται σε ένα data dictionary
- Έλεγχος συμβατότητας (consistency checker): κάθε item στο dictionary έχει οριστεί στις προδιαγραφές και το ανάποδο
- Εύκολη δημιουργία screens και reports
- Πάρα πολλά κάνουν χρήση της UML
- Παραδείγματα: *Analyst/Designer*, *Rose*, *Paradigm Plus*

ΜΕΤΡΙΚΕΣ ΓΙΑ ΤΗ ΦΑΣΗ ΤΗΣ ΣΧΕΔΙΑΣΗΣ

Πέντε βασικές (size, cost, duration, effort, quality)

Ποιότητα

- Συνεκτικότητα
- Σύζευξη

Κυκλωματική πολυπλοκότητα – Cyclomatic complexity

- Για detailed design
- Αριθμός δυαδικών αποφάσεων συν ένα (π.χ. true/false) → στην ουσία ο αριθμός των branches
- Όσο μικρότερη τόσο καλύτερη η σχεδίαση
- Μειονέκτημα: δε λαμβάνει υπόψιν την πολυπλοκότητα data-driven modules, μόνο την πολυπλοκότητα ελέγχου (παράδειγμα μετατροπής text σε ASCII)